Software Engineering Intro

Condensed Study Notes

Generated: 2026-05-07 06:51 UTC

Course Contents

This course will cover several key areas related to software development.

Software Design: This involves understanding how to structure software effectively.

- **Software architecture:** The fundamental organization of a software system.
- **Design patterns:** Reusable solutions to common software design problems.
- **OO analysis & Design (Object-Oriented analysis and Design):** A method of understanding and designing systems by modeling them as a collection of interacting objects.
- **Design for re-use:** Creating software components that can be used in multiple projects.

Using APIs (Application Programming Interfaces): This focuses on how to interact with existing software components.

- **API programming:** Writing code that uses APIs to leverage functionality from other software.
- **Class browsers and Related tools:** Tools that help developers explore and understand the structure of code, particularly classes.
- **Component-based computing:** Building software by assembling independent, interchangeable components.

Software tools and Environment: This section explores the tools used throughout the software development lifecycle.

- **Requirements analysis and design modeling tools:** Software that aids in defining what a system should do and how it should be designed.
- **Testing tools:** Software used to verify that a program functions correctly.
- **Tool integration mechanisms:** Ways in which different software tools can work together.

The Term Software

Software refers to the set of instructions that tell a computer how to take inputs and transform them into the desired outputs, fulfilling specific functions and performance expectations set by the user.

It also encompasses the accompanying documentation, like a software manual, which aids users in understanding how the software system works. Think of software as the 'brain' of a device, providing the logic and instructions for it to perform tasks, much like a recipe guides a chef on how to prepare a meal.

Today’s software is comprised of

Software is made up of several key components that work together.

1. **Source Code**: This is the human-readable set of instructions written by programmers using specific programming languages (like Java, Python, etc.). Think of it as the original recipe.

2. **Executable**: This is the compiled version of the source code. It's a form that the computer can directly understand and run without needing the original source code. This is like the baked cake, ready to be served.

3. **Data Structures**: These are ways to organize and store information so that programs can effectively manage and use it. For example, a list or a table are types of data structures.

4. **Documents**: This includes various types of documentation that describe the software's operation and how to use it. This can range from user guides to technical manuals, and even design documents like UML diagrams or class diagrams that map out the software's structure.

Software also needs to consider the **platform**, which refers to the specific hardware and operating system it's designed to run on. This is why software is often described with its capabilities and the platform requirements.

Once developed, software goes through **installation** on the user's system, making it functional. Following installation is **implementation**, where the software is used for its intended real-world purpose, ensuring it integrates smoothly with existing systems.

The Term Engineering

Engineering is defined as the profession and activity of designing structures like roads and bridges, or machinery and equipment, based on acquired knowledge.

More than just a field of study, engineering is an action-oriented approach to problem-solving. It involves setting goals, working within constraints, and managing processes through measurement and action to achieve desired outcomes.

At its core, engineering means to 'make, achieve, or get through contrivance or guile.'

- **Contrivance** refers to a device, such as a machine, created for a specific function.
- **Guile**, in this context, relates to finding solutions to problems that require cleverness or strategic thinking, not necessarily deceit.

The engineering process involves identifying needs and resources to achieve goals, ultimately producing a product or providing a service.

Distinguishing Source Code and Executables in VB

This section focuses on practically differentiating between source code and executable files specifically within the context of Visual Basic (VB).

Source code is the human-readable set of instructions written by a programmer in a specific programming language, like VB. It's like a recipe written in English that a chef can understand and follow to prepare a dish.

An executable file, often with a .exe extension, is the compiled version of source code. It has been translated into machine code that the computer's processor can directly understand and run. Think of it as the pre-packaged, ready-to-eat meal that a factory produced from the original recipe.

Key differences to observe when differentiating:

- **Readability:** Source code is plain text, while executables are not human-readable.
- **Purpose:** Source code is for writing and editing programs, while executables are for running them.
- **Format:** Source code files (e.g., .vb files in VB) are text-based, whereas executable files are binary.

The Term Software Engineering

Software Engineering is the application of engineering principles to the development of software. It focuses on creating cost-effective solutions to practical problems by using scientific knowledge to build software artifacts that serve people.

This approach is particularly important for larger and more complex software systems. It emphasizes achieving best practices and applying systematic, disciplined, and quantifiable processes throughout the software's lifecycle, including its development, operation, and maintenance.

Essentially, software engineering involves analyzing user needs and then designing, constructing, and testing applications to meet those needs using programming languages. It's about applying engineering rigor to software creation.

Computer Science VS Software Engineering

Computer Science is primarily the theoretical and mathematical study of how computers function.

Software Engineering focuses on the practical aspects of building software systems, covering areas like project management, architecture, risk management, and testing.

Both fields provide foundational programming and computer science knowledge, enabling a path to becoming a software developer.

Choose Computer Science if you enjoy mathematics and logic, or if you are interested in specialized areas like artificial intelligence, machine learning, security, or graphics.

Choose Software Engineering for a hands-on approach to understanding the complete lifecycle of software development and maintenance.

OR

Software Engineering is a discipline focused on creating practical and cost-effective software solutions. It applies scientific knowledge and systematic methods to the design, construction, operation, and maintenance of software.

The ultimate goal is to produce fault-free software that meets user needs, delivered on time and within budget. The term "Software Engineering" itself signifies a movement towards making software development a more structured and systematic process.

The key differences are

Computer Science offers more flexibility with electives in advanced topics. Students can choose from areas like security, machine learning, computer vision, and database management.

Software Engineering, on the other hand, has more required courses in its specific fundamentals. These include essential areas such as software testing, design principles, and how to define software requirements.

Importance of Software

Software is a fundamental driving force in modern society. Its importance can be seen in several key areas:

- **Business Decision Making:** It forms the basis for how businesses make critical decisions.
- **Scientific and Engineering Advancement:** Software is essential for modern scientific research and solving complex engineering problems.
- **Embedded Systems:** It is integrated into a vast array of systems we use daily, including transportation, medical devices, telecommunications, military operations, industrial processes, entertainment, and office products.
- **Pervasive Impact:** Software affects nearly every aspect of our lives, influencing commerce, culture, and everyday activities, making its societal and cultural impact significant.

As the importance of software continues to grow, the software community is dedicated to developing technologies that improve the efficiency, speed, and cost-effectiveness of building high-quality computer programs.